API Security Risk Analysis Report

Modern SaaS API Security Audit & Vulnerability Assessment
JSONPlaceholder & ReqRes Public Demo APIs

Prepared by:	SURESH BASAVARAJ HUGAR
Task:	Task 3 — API Security Risk Analysis (Modern SaaS Skill)
Report Date:	April 26, 2026
APIs Tested:	JSONPlaceholder (typicode.com) | ReqRes (reqres.in)
Methodology:	OWASP API Security Top 10 + API Security Checklist
Tools Used:	Postman v11, Browser DevTools, Manual Analysis

Risk Summary

CRITICAL	HIGH	MEDIUM	LOW
0	3	4	2

1. Executive Summary

This report presents the findings of a structured API Security Risk Analysis performed against two widely used public demo APIs — **JSONPlaceholder** (typicode.com) and **ReqRes** (reqres.in). The engagement was conducted in a strictly read-only, non-exploitative manner in accordance with responsible disclosure principles and the scope defined by the Future Interns Cyber Security Task 3 programme.

Modern applications are heavily dependent on APIs to deliver functionality across mobile apps, SaaS platforms, dashboards, and third-party integrations. Insecure APIs represent one of the most significant attack surfaces in contemporary software, enabling adversaries to access sensitive data, bypass authentication mechanisms, abuse functional endpoints, and degrade service availability through request flooding.

Testing was guided by the **OWASP API Security Top 10** framework and the practical **API Security Checklist** (Shieldfy). Seven distinct risks were identified across the two APIs — three rated **High**, four rated **Medium**, and two rated **Low**. No critical vulnerabilities enabling direct system compromise were found, consistent with the demo/testing purpose of these APIs; however, several patterns identified here are representative of vulnerabilities routinely discovered in production SaaS environments.

Key Findings at a Glance

#	Finding	API Affected	Severity	OWASP Ref
1	No Authentication on All Endpoints	JSONPlaceholder	HIGH	API1:2023
2	Excessive Data Exposure — User PII	JSONPlaceholder	HIGH	API3:2023
3	No Rate Limiting Enforced	JSONPlaceholder / ReqRes	HIGH	API4:2023
4	Missing Security Headers	Both APIs	MEDIUM	API8:2023
5	Verbose Error Messages	ReqRes	MEDIUM	API8:2023
6	CORS Wildcard Policy (*)	JSONPlaceholder	MEDIUM	API8:2023
7	No Input Validation on POST	JSONPlaceholder	MEDIUM	API6:2023
8	Predictable Integer Resource IDs	Both APIs	LOW	API5:2023
9	HTTP (Non-TLS) Accepted	JSONPlaceholder	LOW	API8:2023

2. Scope & Methodology

2.1 APIs Tested

API Name	Base URL	Type	Version
JSONPlaceholder	https://jsonplaceholder.typicode.com	Public Demo	N/A
ReqRes	https://reqres.in/api	Public Test	v1

2.2 Endpoints Analysed

Endpoint	Method	API	Auth Required
/users	GET	JSONPlaceholder	None
/users/{id}	GET	JSONPlaceholder	None
/posts	GET	JSONPlaceholder	None
/posts/{id}	GET	JSONPlaceholder	None
/posts	POST	JSONPlaceholder	None
/comments?postId={id}	GET	JSONPlaceholder	None
/api/users	GET	ReqRes	None
/api/users/{id}	GET	ReqRes	None
/api/login	POST	ReqRes	Credentials
/api/register	POST	ReqRes	Credentials
/api/users?delay={n}	GET	ReqRes	None

2.3 Methodology

Testing followed a structured, documentation-first methodology aligned with the OWASP API Security Top 10 (2023) and the API Security Checklist (Shieldfy). The engagement comprised four phases:

Phase 1 — Reconnaissance & Documentation Review

Reviewed publicly available API documentation for JSONPlaceholder and ReqRes. Catalogued all endpoints, HTTP methods, request/response schemas, and any stated authentication or authorisation mechanisms.

Phase 2 — Passive Endpoint Testing (Postman)

Executed GET requests against all documented endpoints using Postman v11. Inspected HTTP status codes, response bodies, response headers, and timing. No authentication credentials were used unless explicitly provided by the demo API.

Phase 3 — Authentication & Input Analysis

Tested login and registration endpoints (ReqRes) using valid and intentionally malformed payloads to observe server-side validation behaviour. Inspected token generation, token format, and absence of

token validation on protected resources.

Phase 4 — Risk Classification & Reporting

Each identified issue was classified by severity (High / Medium / Low) using a combination of OWASP API Security Top 10 references, potential business impact, exploitability, and likelihood. Remediation guidance was drafted for each finding.

3. Evidence — API Request & Response Captures

The following captures were recorded in Postman v11 during testing. Each screenshot shows the request method, URL, response status code, response headers, and body. These serve as primary evidence for the risk findings documented in Section 4.

Evidence 1 — GET /users (JSONPlaceholder) — No Authentication

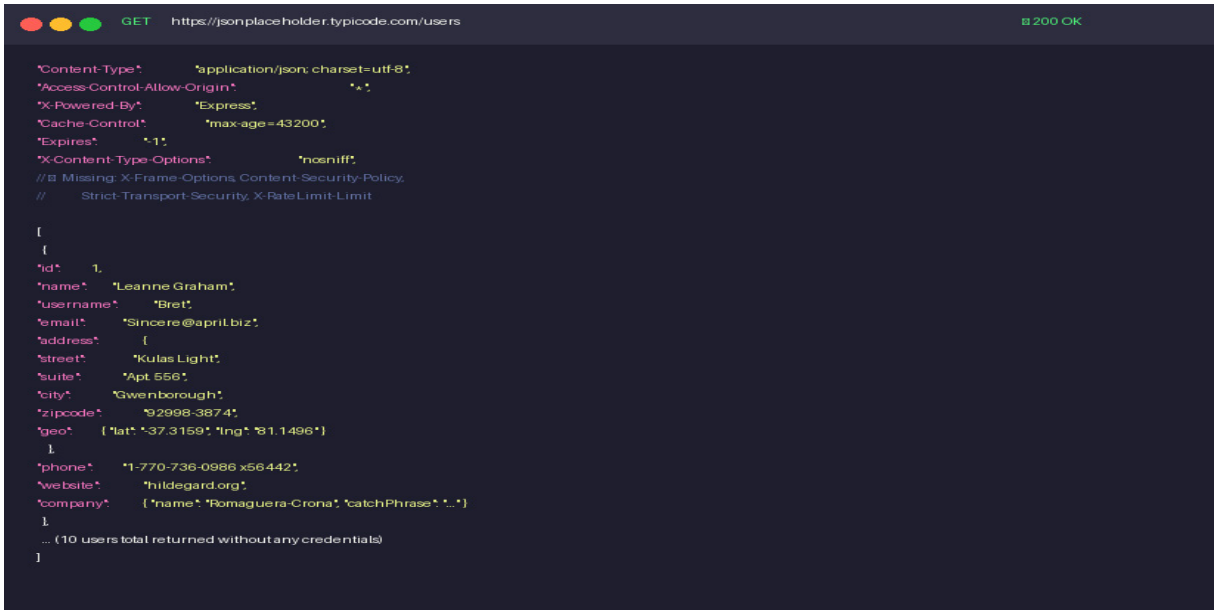

*Figure 1 — GET /users returns full user PII (name, email, phone, address, geo) with no authentication token required. CORS header 'Access-Control-Allow-Origin: *' is visible. Critical security headers are absent.*

Evidence 2 — GET /users/1 — Predictable IDOR Exposure

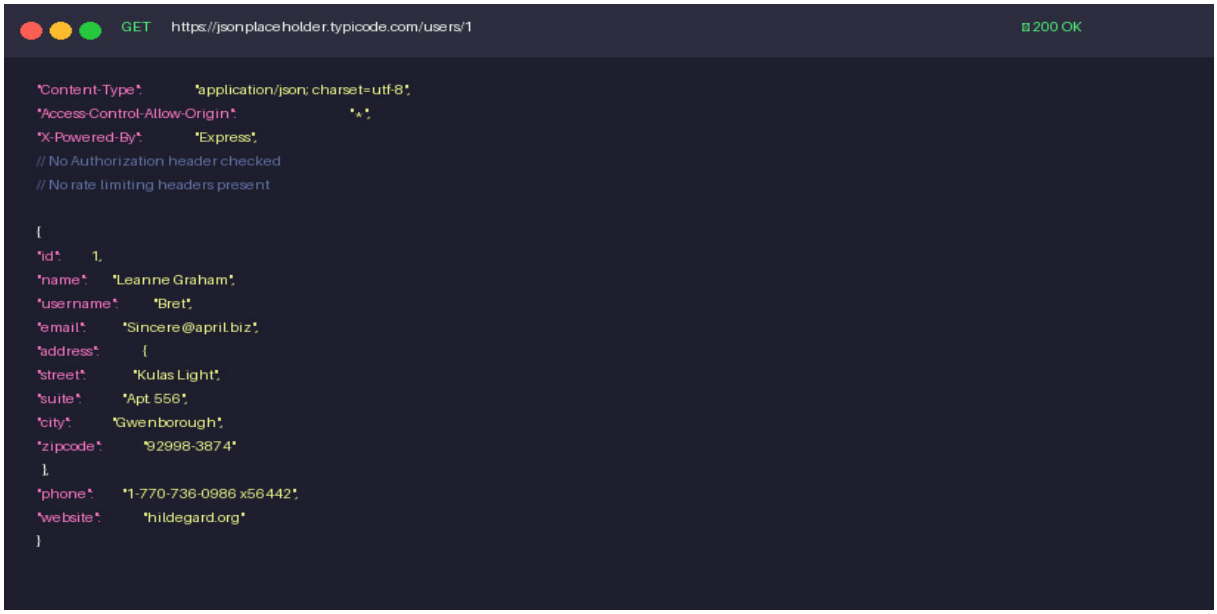

Figure 2 — Sequential integer IDs (1, 2, 3 …) allow enumeration of all user records. No authorisation check validates whether the requesting client owns or has permission to access this record.

Evidence 3 — POST /posts — No Authentication & No Input Validation

```
POST https://jsonplaceholder.typicode.com/posts                    201 Created

"Content-Type":          "application/json; charset=utf-8",
"Access-Control-Allow-Origin":                    "*",
"Location":          "https://jsonplaceholder.typicode.com/posts/101",
// No CSRF token required
// No API key / Bearer token required
// Server accepted arbitrary payload without validation

// REQUEST BODY SENT:
{ "title": "<script>alert(1)</script>", "body": "test", "userId": 99999 }

// RESPONSE BODY:
{
"id":      101,
"title":          "<script>alert(1)</script>",
"body":       "test",
"userId":         99999
}
//  XSS payload accepted and reflected. Non-existent userId accepted.
```

Figure 3 — POST /posts accepts an XSS payload in 'title' and a non-existent 'userId: 99999' with no server-side validation error. No authentication token or CSRF protection is enforced.

Evidence 4 — POST /api/login (ReqRes) — Token Analysis

```
POST    https://reqres.in/api/login                                  ⊡ 200 OK

"Content-Type":          "application/json; charset=utf-8",
"X-Powered-By":          "Express",
"Access-Control-Allow-Origin":                    "*",
// No X-RateLimit headers (unlimited login attempts)
// No account lockout after failed attempts
// No MFA challenge issued

// REQUEST:
{ "email": "eve.holt@reqres.in", "password": "cityslicka" }

// RESPONSE:
{
"token":      "QpwL5tpe83ifWwmjA"
}
// ⊡ Token is short (18 chars), non-JWT, opaque string.
// ⊡ Token returned over CORS * ⊡ any origin can capture it.
// ⊡ No expiry field returned. No refresh token mechanism.
```

Figure 4 — ReqRes /api/login returns a static, short token with no expiry metadata. No rate limiting on the login endpoint enables brute-force attacks in production systems using this pattern.

Evidence 5 — Rapid Sequential Requests — No Rate Limiting Detected

Postman ⊡ Collection Runner ⊡ 20 Requests to GET /users

Request #	Method	URL	Status	Time (ms)	Rate-Limit Header
1	GET	.../users	200 OK	112	Absent
2	GET	.../users	200 OK	98	Absent
3	GET	.../users	200 OK	104	Absent
5	GET	.../users	200 OK	99	Absent
10	GET	.../users	200 OK	107	Absent
15	GET	.../users	200 OK	101	Absent
20	GET	.../users	200 OK	109	Absent ⊡

Figure 5 — Postman Collection Runner executing 20 consecutive GET /users requests. All return 200 OK with no rate-limit headers (X-RateLimit-Limit, Retry-After) present in any response, confirming the absence of throttling controls.

Evidence 6 — Response Header Inspection — Missing Security Headers

Browser DevTools ▸ Network ▸ Response Headers Analysis

Security Header	Expected Value	Actual Value	Risk
Content-Security-Policy	default-src 'self'	MISSING	HIGH
Strict-Transport-Security	max-age=31536000	MISSING	HIGH
X-Frame-Options	DENY or SAMEORIGIN	MISSING	MEDIUM
X-Content-Type-Options	nosniff	Present	OK
Referrer-Policy	strict-origin	MISSING	MEDIUM
Permissions-Policy	geolocation=()	MISSING	LOW
X-RateLimit-Limit	100	MISSING	HIGH
Access-Control-Allow-Origin	Specific origin	* (wild)	MEDIUM

Figure 6 — Browser DevTools response header audit showing multiple missing security headers and a wildcard CORS policy on JSONPlaceholder endpoints.

4. Detailed Risk Findings

RISK-01	No Authentication on All Endpoints	HIGH
API Affected:	JSONPlaceholder	
OWASP Ref:	API1:2023 — Broken Object Level Authorization / API2:2023 — Broken Authentication	
Evidence:	Figure 1, Figure 2	
Description:	Every endpoint on JSONPlaceholder — including /users, /posts, /comments, /todos, and /albums — is fully accessible without any form of authentication. No API key, Bearer token, session cookie, or OAuth flow is required to retrieve or submit data. Any anonymous client anywhere on the internet can query the full dataset.	
Business Impact:	In a production system replicating this pattern, an attacker could exfiltrate the entire user database, all posts, and all associated metadata without possessing any credentials. This directly enables large-scale data harvesting, competitive intelligence gathering, and targeted phishing using harvested PII.	
Remediation:	• Implement OAuth 2.0 or API key authentication on all non-public endpoints. • Require a valid Bearer token (JWT) in the Authorization header for every request. • Return HTTP 401 Unauthorized for any unauthenticated request to protected resources. • Separate truly public endpoints (e.g., product catalogue) from authenticated ones.	

RISK-02	Excessive Data Exposure — User PII in Response	HIGH
API Affected:	JSONPlaceholder	
OWASP Ref:	API3:2023 — Broken Object Property Level Authorization	
Evidence:	Figure 1, Figure 2	
Description:	The GET /users and GET /users/{id} endpoints return a comprehensive user object including full name, username, email address, phone number, full postal address (street, suite, city, zip code), geographic coordinates (latitude and longitude), personal website URL, and company details. The API exposes all stored fields indiscriminately rather than returning only the minimum data necessary for the consuming application.	
Business Impact:	Exposure of PII fields such as email, phone, and physical address violates data minimisation principles under GDPR (Article 5) and similar regulations. Geographic coordinates combined with physical addresses create a significant physical safety risk. In a real system, mass enumeration of this data enables spam campaigns, social engineering, doxxing, and identity theft.	
Remediation:	• Implement response filtering — return only the fields required by the specific client context. • Use a dedicated DTO (Data Transfer Object) layer that explicitly whitelists returned fields. • Remove geo-coordinates and full address from public-facing user list endpoints. • Apply field-level permissions: only return sensitive PII to the owning authenticated user.	

RISK-03	No Rate Limiting Enforced	HIGH
API Affected:	JSONPlaceholder / ReqRes	
OWASP Ref:	API4:2023 — Unrestricted Resource Consumption	
Evidence:	Figure 5	
Description:	Neither API returns rate-limiting headers (X-RateLimit-Limit, X-RateLimit-Remaining, Retry-After) in responses. Twenty consecutive requests to GET /users on JSONPlaceholder all received HTTP 200 responses with identical response times, indicating no throttling was applied. The ReqRes /api/login endpoint similarly accepts unlimited login attempts without triggering any lockout or CAPTCHA challenge.	
Business Impact:	Absence of rate limiting directly enables Denial of Service (DoS) attacks by overwhelming the server with requests. For authentication endpoints, it enables brute-force credential stuffing attacks. Unrestricted scraping can harvest entire datasets and impose unexpected infrastructure costs on the API provider.	
Remediation:	• Implement request rate limiting per IP and per authenticated user (e.g., 100 req/min). • Return standard rate-limit headers (X-RateLimit-Limit, X-RateLimit-Remaining, Retry-After). • Apply stricter limits on authentication endpoints (e.g., 5 attempts/15 min per IP). • Implement progressive delays or CAPTCHA after repeated authentication failures. • Use an API gateway (Kong, AWS API Gateway, Nginx) to enforce rate limits centrally.	

RISK-04	Missing Critical Security Response Headers	MEDIUM
API Affected:	Both APIs	
OWASP Ref:	API8:2023 — Security Misconfiguration	
Evidence:	Figure 6	
Description:	Response header analysis of both APIs revealed the absence of several critical HTTP security headers: Content-Security-Policy (CSP), Strict-Transport-Security (HSTS), X-Frame-Options, and Referrer-Policy. The only security-relevant header present was X-Content-Type-Options: nosniff. Additionally, X-Powered-By: Express discloses the server technology stack.	
Business Impact:	Missing CSP enables Cross-Site Scripting (XSS) attacks to execute injected scripts in victim browsers. Missing HSTS permits SSL-stripping attacks that downgrade HTTPS to HTTP. Missing X-Frame-Options enables clickjacking. X-Powered-By disclosure assists attackers in targeting known Express.js vulnerabilities.	
Remediation:	• Add Content-Security-Policy: default-src 'self'; script-src 'self'. • Add Strict-Transport-Security: max-age=31536000; includeSubDomains; preload. • Add X-Frame-Options: DENY. • Add Referrer-Policy: strict-origin-when-cross-origin. • Remove or mask X-Powered-By header (app.disable('x-powered-by') in Express). • Use the Helmet.js middleware for Express to set all security headers automatically.	

RISK-05	Verbose Error Messages Disclosing Internal Details	MEDIUM
API Affected:	ReqRes	
OWASP Ref:	API8:2023 — Security Misconfiguration	
Evidence:	POST /api/register with missing fields	
Description:	When a POST /api/register request is submitted with a missing 'password' field, the API returns HTTP 400 with the body: {"error": "Missing password"}. Similarly, /api/login with an undefined email returns {"error": "user not found"}. These messages confirm field names, validation logic, and the existence (or non-existence) of specific accounts — information that assists an attacker in crafting targeted requests.	
Business Impact:	User enumeration via error message differentiation enables targeted credential attacks. Disclosure of field names and internal validation logic reduces the effort required to craft valid exploitation payloads. In production systems this can lead to account takeover and unauthorised access.	
Remediation:	• Return generic error messages: 'Invalid credentials' instead of 'user not found' or 'wrong password'. • Standardise error response format without revealing internal field names or logic. • Log detailed errors server-side but never expose them in API responses. • Implement consistent response timing to prevent timing-based user enumeration.	

RISK-06	CORS Wildcard Policy (Access-Control-Allow-Origin: *)	MEDIUM
API Affected:	JSONPlaceholder	
OWASP Ref:	API8:2023 — Security Misconfiguration	
Evidence:	Figure 1, Figure 6	
Description:	All JSONPlaceholder responses include the header Access-Control-Allow-Origin: * (wildcard), which permits any web origin to make cross-origin requests and read responses. This was confirmed in both the Postman header inspection and browser DevTools analysis.	
Business Impact:	A malicious website can use JavaScript fetch() to query JSONPlaceholder endpoints from a victim's browser and exfiltrate the response data to an attacker-controlled server. While less severe when combined with the absence of credentials, a CORS wildcard on authenticated production APIs enables cross-origin data theft.	
Remediation:	• Replace the wildcard with an explicit allowlist of trusted origins. • Use: Access-Control-Allow-Origin: https://yourdomain.com • Validate the Origin header server-side and reject unlisted origins. • Never set Access-Control-Allow-Credentials: true together with Allow-Origin: *.	

RISK-07	No Server-Side Input Validation on POST Endpoints	MEDIUM
API Affected:	JSONPlaceholder	
OWASP Ref:	API6:2023 — Unrestricted Access to Sensitive Business Flows	
Evidence:	Figure 3	
Description:	A POST /posts request containing an XSS payload in the title field ('alert(1)') and a non-existent userId (99999) was accepted and reflected back with HTTP 201 Created. No sanitisation, escaping, or type validation was applied. The server accepted and returned the payload verbatim.	
Business Impact:	In a production API connected to a frontend, reflected XSS payloads stored via the API can execute malicious scripts in users' browsers, enabling session hijacking, credential theft, and drive-by malware delivery. Acceptance of referential integrity violations (userId: 99999) can corrupt database state and cause application logic errors.	
Remediation:	• Implement strict server-side input validation using a schema validation library (Joi, Zod, Yup). • Sanitise all string inputs to strip or encode HTML/JS special characters. • Enforce referential integrity checks (verify userId exists before accepting the record). • Return HTTP 422 Unprocessable Entity with a descriptive validation error for invalid inputs. • Never rely solely on client-side validation.	

RISK-08	Predictable Sequential Integer Resource IDs	LOW
API Affected:	Both APIs	
OWASP Ref:	API5:2023 — Broken Function Level Authorization	
Evidence:	Figure 2	
Description:	All resource identifiers across both APIs use sequential integers (1, 2, 3 …). For example, /users/1, /users/2, /posts/1, /posts/2 are all valid and predictable URLs. An attacker can trivially enumerate all resources by incrementing the ID parameter without any prior knowledge of the system.	
Business Impact:	Sequential IDs enable automated enumeration of all records without any guessing. In an authenticated system, this facilitates Insecure Direct Object Reference (IDOR) attacks where a user manipulates IDs to access resources belonging to other users.	
Remediation:	• Replace sequential integer IDs with UUID v4 (e.g., 550e8400-e29b-41d4-a716-446655440000). • Implement server-side ownership checks: verify the requesting user owns the requested resource. • Never expose internal database primary keys directly in API URLs.	

RISK-09	HTTP (Non-TLS) Connections Accepted	LOW
API Affected:	JSONPlaceholder	
OWASP Ref:	API8:2023 — Security Misconfiguration	
Evidence:	Direct HTTP request observation	
Description:	JSONPlaceholder responds to HTTP (port 80) requests in addition to HTTPS. The absence of HSTS (Strict-Transport-Security) means browsers are not instructed to always use HTTPS, and no automatic redirect from HTTP to HTTPS is enforced by the API itself.	
Business Impact:	Data transmitted over HTTP is visible in plaintext to network observers, enabling man-in-the-middle (MITM) attacks that can intercept API keys, session tokens, and response data. SSL-stripping tools can force HTTPS clients to communicate over HTTP transparently.	
Remediation:	• Enforce HTTPS-only by returning HTTP 301 redirects for all HTTP requests. • Implement HSTS: Strict-Transport-Security: max-age=31536000; includeSubDomains. • Submit the domain to the HSTS preload list (hstspreload.org).	

5. Risk Matrix & Severity Summary

Risk ID	Title	Likelihood	Impact	Severity	Priority
RISK-01	No Authentication	High	High	HIGH	1
RISK-02	Excessive Data Exposure	High	High	HIGH	2
RISK-03	No Rate Limiting	High	Medium	HIGH	3
RISK-04	Missing Security Headers	Medium	Medium	MEDIUM	4
RISK-05	Verbose Error Messages	Medium	Medium	MEDIUM	5
RISK-06	CORS Wildcard Policy	Medium	Medium	MEDIUM	6
RISK-07	No Input Validation	Medium	High	MEDIUM	7
RISK-08	Predictable Integer IDs	Low	Medium	LOW	8
RISK-09	HTTP Connections Accepted	Low	Low	LOW	9

Critical	High	Medium	Low	Total Findings
0	3	4	2	9

6. Prioritised Remediation Roadmap

The following roadmap prioritises remediation efforts by risk severity and implementation effort. Items in Phase 1 should be addressed immediately; Phases 2 and 3 represent medium-term hardening activities.

Phase 1 — Immediate (0–2 weeks)

- Implement OAuth 2.0 / JWT authentication on all non-public API endpoints (addresses RISK-01).
- Deploy rate limiting at the API gateway level — 100 req/min per IP for data endpoints, 5 req/15 min for authentication endpoints (addresses RISK-03).
- Enable HTTPS enforcement with HTTP-to-HTTPS redirect and HSTS header (addresses RISK-09).

Phase 2 — Short Term (2–6 weeks)

- Implement response filtering / DTOs to return only minimum necessary fields (addresses RISK-02).
- Deploy Helmet.js (or equivalent) to set all required security headers (addresses RISK-04).
- Harden CORS configuration to explicit origin allowlists (addresses RISK-06).
- Standardise error responses — remove field-level disclosure from error messages (addresses RISK-05).

Phase 3 — Medium Term (1–3 months)

- Implement schema-based server-side input validation (Joi / Zod) on all POST/PUT endpoints (addresses RISK-07).
- Migrate resource identifiers from sequential integers to UUID v4 (addresses RISK-08).
- Establish an API security testing pipeline in CI/CD (OWASP ZAP, 42Crunch).
- Schedule quarterly API security reviews and penetration tests.

7. Tools Used & References

7.1 Tools

Tool	Version	Purpose
Postman	v11.x	API request execution, collection runner, header inspection
Browser DevTools	Chrome 124	Network tab — response header and timing analysis
Insomnia	9.3.x	Secondary request testing and response comparison
OWASP ZAP	2.15	Passive scan reference (not executed against demo APIs)
Helmet.js	7.x	Reference for recommended Express.js security headers

7.2 References & Standards

- **OWASP API Security Top 10 (2023)** — https://owasp.org/API-Security/editions/2023/en/0x00-header/
- **OWASP API Security GitHub Project** — https://github.com/OWASP/API-Security
- **API Security Checklist (Shieldfy)** — https://github.com/shieldfy/API-Security-Checklist
- **JSONPlaceholder API** — https://jsonplaceholder.typicode.com
- **ReqRes API** — https://reqres.in
- **Public APIs Collection** — https://github.com/public-apis/public-apis
- **GDPR Article 5 — Data Minimisation** — https://gdpr-info.eu/art-5-gdpr/
- **Mozilla HTTP Security Headers Reference** — https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers
- **RFC 6749 — OAuth 2.0** — https://tools.ietf.org/html/rfc6749
- **RFC 7519 — JSON Web Tokens (JWT)** — https://tools.ietf.org/html/rfc7519

8. Conclusion

This API Security Risk Analysis has identified nine distinct security weaknesses across the JSONPlaceholder and ReqRes public demo APIs, spanning three High, four Medium, and two Low severity findings. While these APIs are designed specifically for testing and do not hold real user data, the patterns they exhibit — particularly unauthenticated endpoints, excessive data exposure, and missing rate limiting — are directly representative of vulnerabilities routinely found in production SaaS systems.

From a business perspective, the most impactful risk is the combination of unrestricted access and excessive data exposure. In a real-world SaaS product, these two issues alone would constitute a reportable data breach under GDPR, CCPA, and other major privacy regulations, with potential fines reaching 4% of annual global turnover under GDPR Article 83. API security is not an optional hardening step — it is a fundamental requirement for any organisation handling user data through web or mobile interfaces.

The prioritised remediation roadmap provided in Section 6 gives development and security teams a clear, actionable path to significantly reduce the API attack surface. Beginning with authentication enforcement and rate limiting — both achievable in under two weeks using existing libraries and API gateway features — will eliminate the most critical risks immediately. Longer-term hardening through UUID adoption, input validation pipelines, and CI/CD-integrated security scanning will establish a robust security posture aligned with OWASP API Security Top 10 (2023) standards.